PROJECT CASE STUDY
CONFIDENTIAL ENTERPRISE INFRASTRUCTURE PROJECT

Nisal G.
AWS Cloud Infrastructure & DevOps Engineer

AWS Cloud Infrastructure & DevOps – Project Case Study

Terraform · Amazon ECS on Fargate · RDS PostgreSQL · CloudFront · Bitbucket Pipelines with OIDC

IMPLEMENTED In Terraform or pipeline code; dev environment deployed.

DOCUMENTED In project docs; source in a repo not reviewed here.

PLANNED Designed, not built yet. Dashed outlines.

1 Executive summary

I designed and built an AWS platform foundation for a containerised business application, defined end to end in **Terraform** and deployed through **keyless CI/CD**. Each environment gets its own AWS account, built from the same modules. The **development environment is implemented and running**: an API, background worker and web admin app on **ECS Fargate** behind CloudFront and an internal ALB, with **RDS PostgreSQL**. Staging, production, WAF and disaster recovery are designed for a later phase.

17
reusable modules; one root module + state per env

4
Fargate services (ARM64): API, worker, scheduler, admin

3 × 2
subnet tiers × AZs; data tier has no internet route

0
static AWS keys in CI/CD: OIDC for every pipeline step

2 Project overview & technical objectives

- **No public origin.** Traffic enters only through CloudFront, which reaches an *internal* ALB through a VPC origin.
- **Account per environment** keeps the blast radius small, billing clean and guardrails separate.
- **No IAM users or static keys.** Engineers use IAM Identity Center with MFA; pipelines use OIDC.
- **Git is the only change path.** Every change is planned in CI, the plan is reviewed, then applied.
- **Secrets stay out of code** and out of Terraform state.
- **Cost-aware non-production:** office-hours scheduling, NAT only where needed, budget alerts.

3 AWS services & technologies

COMPUTE & CONTAINERS	Amazon ECS · Fargate (ARM64) · Amazon ECR · AWS Lambda · ECS Exec
NETWORKING & EDGE	Amazon VPC · ALB (internal) · CloudFront + VPC origins · ACM · S3 gateway endpoint · Route 53
DATA	RDS for PostgreSQL · Amazon S3
SECURITY & IDENTITY	AWS KMS · Secrets Manager · IAM + OIDC federation · IAM Identity Center · AWS Organizations
OPERATIONS	CloudWatch Logs & Alarms · Amazon SNS · EventBridge Scheduler · AWS Budgets
TOOLING	Terraform ≥ 1.11 · Bitbucket Pipelines · Docker (multi-stage, distroless) · Trivy · SBOM
PLANNED	AWS WAF · Control Tower · GuardDuty · Security Hub · Org CloudTrail · Cross-Region backups

4 Terraform infrastructure as code

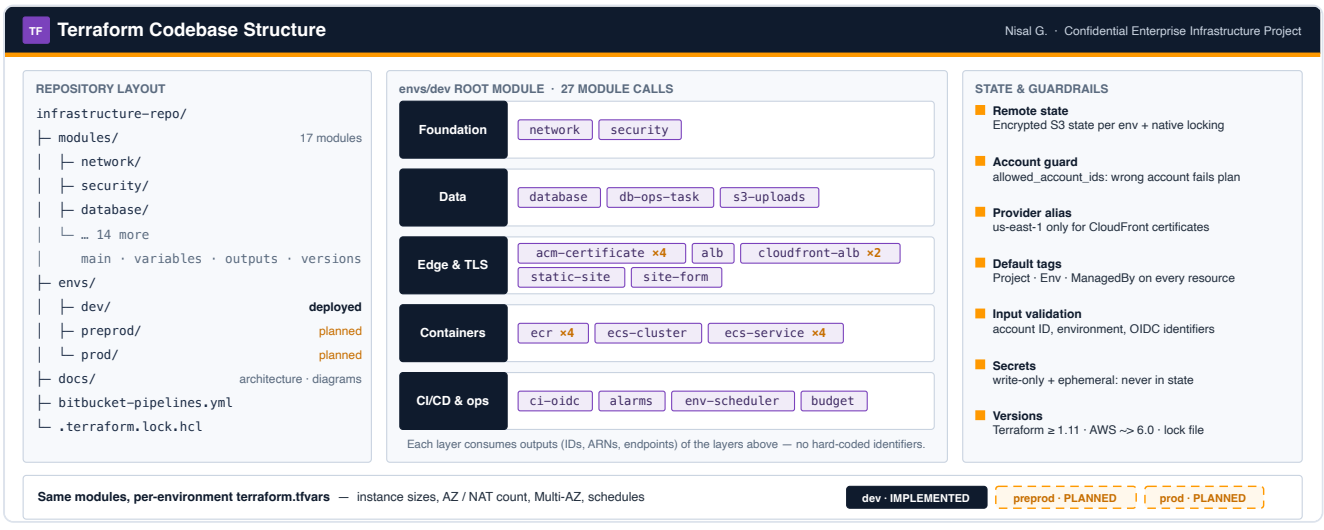


Diagram 1. Repository layout, module composition and state guardrails (no source code shown). `moved` blocks and `create_before_destroy` protect live resources during refactors and upgrades.

5 AWS architecture

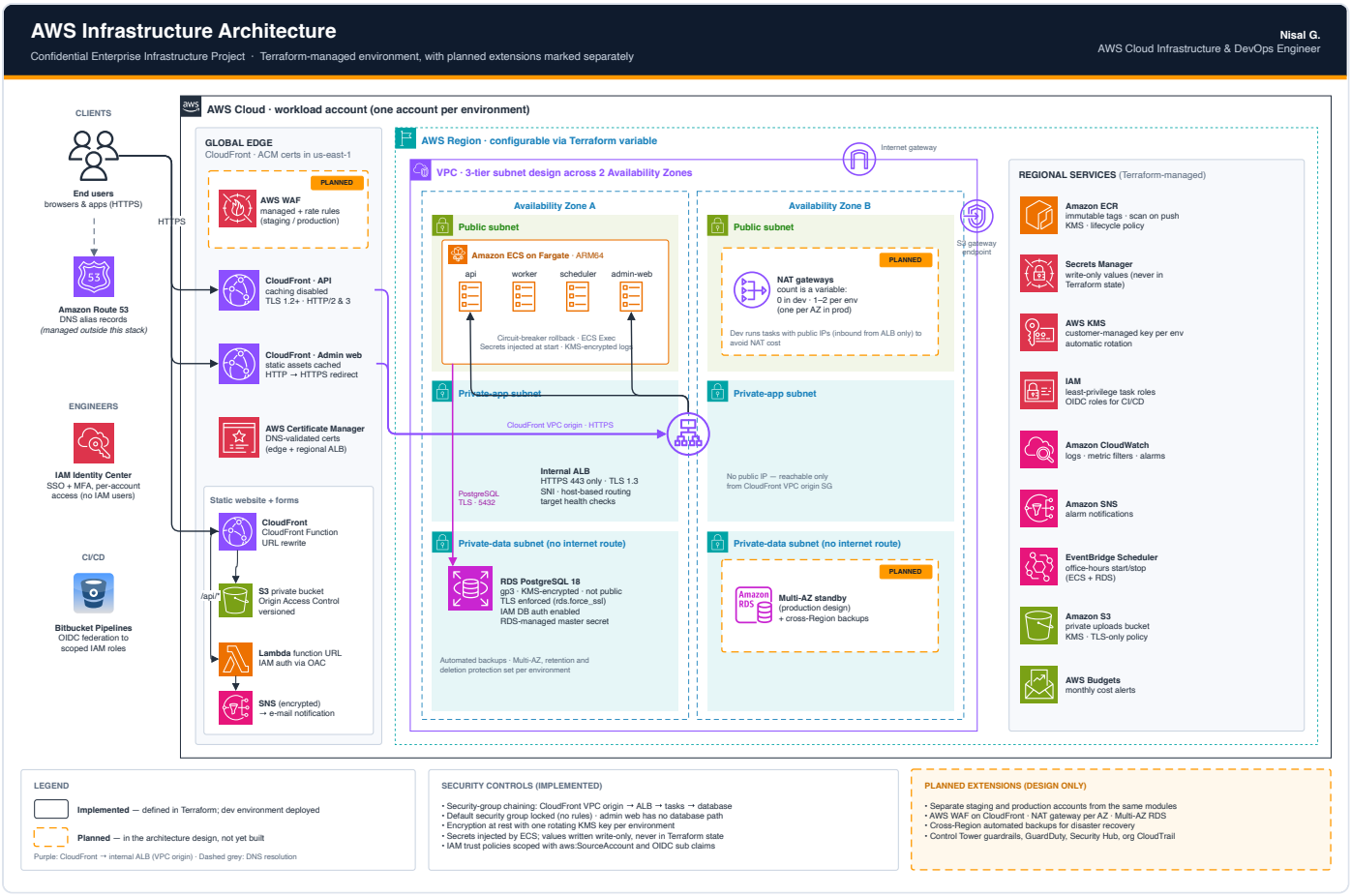


Diagram 2. Solid = implemented in Terraform; dashed orange = planned. In dev, tasks use public IPs for outbound traffic, with inbound allowed from the ALB only. This is a documented cost decision (no NAT gateway).

6 Security & networking design

Network

- The VPC has **public, private-app and private-data** subnets across 2 AZs. The data tier has no internet route.
- The **internal ALB** has no public IP and accepts traffic only from the CloudFront VPC-origin security group: HTTPS only, TLS 1.3 policy, invalid headers dropped, host routing with SNI.
- **CloudFront** requires TLS 1.2+; API traffic is never cached, static assets are.
- **NAT count is a variable** (0 in dev, one per AZ in the production design). An S3 gateway endpoint keeps S3 traffic inside AWS.

Security controls

- **Security-group chaining:** CloudFront → ALB → tasks → database. The admin web has no database path; the default security group is emptied.
- **One rotating KMS key** per environment for RDS, Secrets Manager, logs, ECR and S3.
- **RDS:** private, TLS enforced (`rds.force_ssl`), IAM auth enabled; the master password is managed and rotated by RDS.
- **S3:** public access blocked, ACLs off, TLS-only policy. The site bucket is readable only by its own distribution (OAC + `SourceArn`).
- **IAM:** separate execution and task roles per service, trust scoped with `aws:SourceAccount` ; ECS Exec instead of SSH or a bastion host.

7 Monitoring & operational practices

- **CloudWatch alarms → SNS e-mail:** ALB 5xx (target and load balancer), unhealthy targets, RDS CPU and free storage.
- **Log-based alerting:** JSON metric filters count `ERROR` lines per service. Missing data counts as OK, so scheduled shutdowns raise no alarms.
- **Logs:** KMS-encrypted log group per service with set retention; PostgreSQL logs queries slower than 1 s.
- **Safe rollouts:** ECS deployment circuit breaker with automatic rollback, ALB health checks and graceful shutdown.
- **Cost operations:** EventBridge Scheduler runs dev on weekdays only (database up first, down last). AWS Budgets alerts at 50 / 80 / 100 % of spend and on forecast overrun.
- **Operational tooling:** one-off ECS tasks run migrations and admin SQL with the RDS-managed secret. No bastion host is needed.

8 CI/CD pipeline architecture & automation

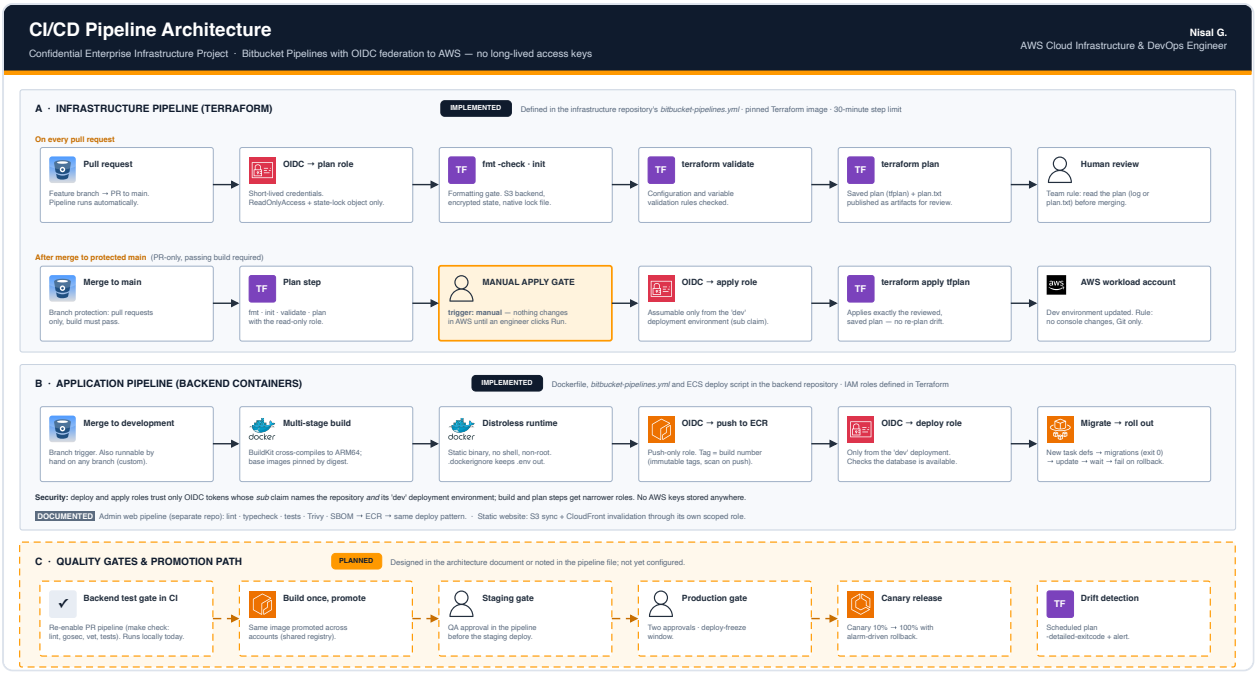


Diagram 3. **Lane A** comes from the infrastructure repo's pipeline file: PR checks run under a read-only OIDC role, and after merge a **manual-trigger step** applies exactly the reviewed plan. **Lane B** comes from the backend repo's Dockerfile, pipeline file and deploy script. **Lane C** is designed but not yet configured.

9 My responsibilities & contributions

- **Architecture owner:** wrote the AWS DevOps architecture document (accounts, identity, network, environments, security, DR, release path, cost).
- **Terraform build:** reusable module library, dev root module, remote state, account guards.
- **Infrastructure CI/CD:** Bitbucket OIDC trust with separate push, deploy, plan and apply roles; the Terraform pipeline with its manual apply gate.
- **Containers & app CI/CD:** wrote the backend Dockerfile (multi-stage ARM64 build, digest-pinned distroless non-root image) and its OIDC build, push and deploy pipeline.
- **Platform:** ECR, ECS Fargate, internal ALB, CloudFront VPC origins, ACM, RDS PostgreSQL (including a major-version upgrade), static site and form.
- **Operations:** alarms, log metrics, budgets, scheduling and runbooks in the repository docs.

10 Relevance to your AWS & DevOps requirements

YOUR REQUIREMENT	DEMONSTRATED IN THIS PROJECT	STATUS
Terraform & AWS provisioning in ap-southeast-1	Region is a variable and no module hard-codes one; CloudFront's us-east-1 certificates use a provider alias.	IMPLEMENTED
Containerised workloads on ECS Fargate	Four ARM64 services, secrets injection, ECS Exec, circuit-breaker rollback, internal ALB.	IMPLEMENTED
PostgreSQL database infrastructure	Private, encrypted RDS PostgreSQL: TLS, IAM auth, managed secret, backups.	IMPLEMENTED
Secure networking & IAM	Three-tier VPC, security-group chaining, KMS, Secrets Manager, OIDC roles, SSO with MFA.	IMPLEMENTED · WAF PLANNED
Staging & production environments	Account per environment, same modules. Dev is built; staging and production are designed.	DEV · STG / PROD
CI/CD, automated testing & approval gates	Terraform fmt / validate / plan and a manual apply gate. Backend pipeline builds, pushes, migrates and rolls out with rollback detection. A CI test gate and QA / two-approval gates are planned.	IAC · APPS · GATES
Security & observability	Encryption at rest, CloudWatch alarms, log metric filters, SNS and budget alerts.	IMPLEMENTED

Service scope

I provide the **AWS infrastructure and DevOps layer**: Terraform, networking, ECS Fargate, RDS PostgreSQL, security, CI/CD and observability. **AI/LLM application development, MCP implementation and on-premises LLM integration are outside my proposed scope.** This project involved no AI technologies. It was a single-organisation platform, not a multi-tenant SaaS, so the tenant-isolation approach for your platform would be agreed during discovery.